

2교시: 미니 앱 skeleton 생성

수업 목표

- 미니 앱의 최소 파일 구조를 만든다.
- 각 파일의 책임을 설명한다.
- 아직 기능을 완성하지 않고도 실행 가능한 빈 skeleton을 확인한다.

50분 운영

시간	활동	학습 초점	학생 산출
0-5분	이전 scope 확인	범위 초과 기능을 다시 제거한다.	확정 scope
5-15분	파일 책임 설명	HTML/CSS/JS/JSON/README 역할을 분리한다.	책임 메모
15-30분	skeleton 작성	최소 markup과 연결 태그를 확인한다.	파일 5개
30-40분	로컬 서버 실행	경로와 port를 확인한다.	첫 실행 evidence
40-50분	구조 점검	파일 누락과 연결 오류를 확인한다.	file tree 캡처 또는 텍스트

0-5분 이전 scope 확인

- 초점: 범위 초과 기능을 다시 제거한다.
- 학생 산출: 확정 scope

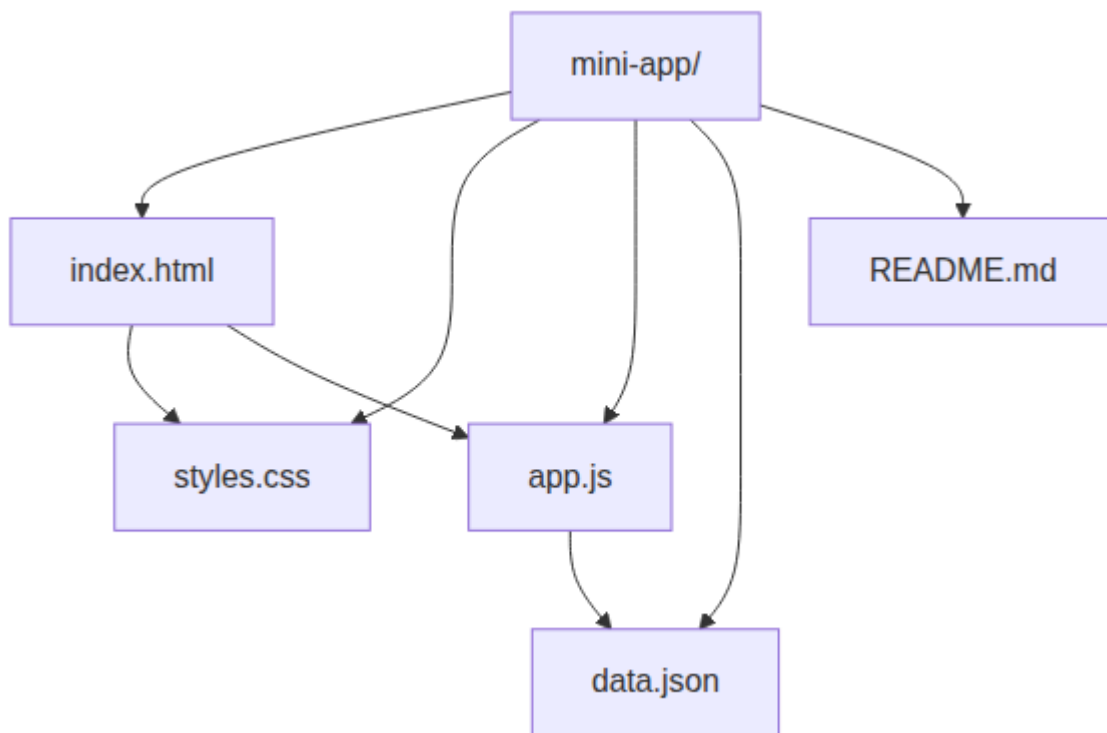
핵심 설명

Skeleton은 "나중에 채울 빈 껍데기"가 아니라 팀원이 구조를 이해할 수 있게 만드는 첫 artifact다. 파일 이름, 폴더 위치, 실행 명령, 데이터 파일 위치가 안정적이면 다음 구현과 검증이 쉬워진다.

Visual 1: 구조 다이어그램



이 이미지는 미니 앱을 파일 이름 목록이 아니라 책임 분리 구조로 보게 한다. HTML은 구조, CSS는 표현, JS는 동작, JSON은 데이터, README는 실행 기록이라는 기준을 먼저 고정한다.



5-15분 파일 책임 설명

- 초점: HTML/CSS/JS/JSON/README 역할을 분리한다.
- 학생 산출: 책임 메모

Skeleton 예시

```
<!doctype html>
<html lang="ko">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Week 1 Mini App</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <main>
    <h1>Week 1 Mini App</h1>
    <section id="app" aria-live="polite">Loading...</section>
  </main>
  <script src="app.js"></script>
</body>
</html>
```

```
[
  { "name": "Item A", "status": "ready" },
  { "name": "Item B", "status": "blocked" },
```

```
{ "name": "Item C", "status": "ready" }
]
```

Visual 2: 파일 책임 표

파일	책임	첫 점검 방법
index.html	화면 뼈대와 연결 태그	browser에서 제목이 보이는지 확인
styles.css	읽기 쉬운 기본 스타일	CSS 파일명을 HTML link와 비교
app.js	데이터 읽기와 렌더링 준비	script 경로와 console 오류 확인
data.json	dummy item 3개 이상	JSON 배열 문법 확인
README.md	start/check/stop 자리	실행 명령을 적을 공간 확인

15-30분 skeleton 작성

- 초점: 최소 markup과 연결 태그를 확인한다.
- 학생 산출: 파일 5개

Visual 3: 첫 실행 캡처 가이드

캡처/기록	확인할 내용
file tree	5개 파일이 같은 폴더에 있는가
browser 화면	제목 또는 placeholder가 보이는가
terminal path	정적 서버가 mini-app 기준으로 실행되는가

권장 파일 구조

mini-app/ index.html styles.css app.js data.json README.md

활동 절차

1. mini-app 폴더를 만든다.
2. index.html 에 styles.css 와 app.js 를 연결한다.
3. data.json 에는 최소 3개 item을 넣는다.
4. README.md 에는 start/check/stop 제목만 먼저 만든다.
5. 정적 서버를 실행하고 browser 또는 curl -I 로 HTTP 응답을 확인한다.

30-40분 로컬 서버 실행

- 초점: 경로와 port를 확인한다.
- 학생 산출: 첫 실행 evidence

흔한 오해

오해	교정
산출물이 있으면 evidence는 나중에 채워도 된다.	evidence는 산출물의 일부다. command, path, status, log, note가 함께 있어야 평가 가능하다.
Week1에서 모든 기술을 깊게 익혀야 한다.	Week1은 컴퓨팅 spine과 운영 증거를 만드는 주차이며, 깊은 hands-on은 각 기술 주차에서 진행한다.

막힌 내용을 숨기는 것이 좋다.

blocker를 증상, 시도한 일, 다음 조치로 기록하는 것이 현업식 진행 관리다.

40-50분 구조 점검

- 초점: 파일 누락과 연결 오류를 확인한다.
- 학생 산출: file tree 캡처 또는 텍스트

산출물

- mini-app 파일 tree
- 연결된 index.html
- 최소 data.json
- 첫 실행 명령과 URL

평가 기준

기준	충족
파일 5개가 정해진 위치에 있다.	
HTML에서 CSS와 JS가 연결된다.	
data.json이 유효한 JSON 배열이다.	
정적 서버로 index page가 열린다.	

현업 DevOps insight

Repository 구조는 운영 문서의 일부다. 파일 위치가 예측 가능하면 자동화, 리뷰, 인수인계가 쉬워진다. 초기 skeleton에서 실행 명령까지 함께 고정하는 습관은 나중에 CI/CD에서도 그대로 이어진다.

학술 근거

- Scaffolding: 초보자가 전체 문제를 한 번에 풀지 않도록 구조를 먼저 제공한다.
- CS2023: 구현 산출물을 모듈과 데이터 파일로 분리한다.
- Situated learning: 실제 개발자처럼 repository와 실행 명령을 함께 다룬다.

다음 주차 연결

Week2에서는 이 파일 구조가 Docker build context가 된다. 파일 위치가 안정적이어야 container image에 무엇을 복사할지 결정할 수 있다.

다음 연결

다음 교시는 skeleton에 HTML 구조, CSS, JS, dummy JSON 연결을 구현한다.

공식/학술 근거 링크

- GitHub Docs: About READMEs, <https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes> - 프로젝트 구조와 시작 방법을 문서화하는 기준이다.
- MIT Missing Semester, <https://missing.csail.mit.edu/> - shell, Git, filesystem 구조를 개발자 기본 역량으로 다루는 근거다.
- Monash Constructive Alignment, <https://www.monash.edu/learning-teaching/teachhg/Teaching-practices/learning-outcomes/how-to/constructive-alignment> - skeleton, 실행, evidence, 평가 기준을 맞추는 기준이다.